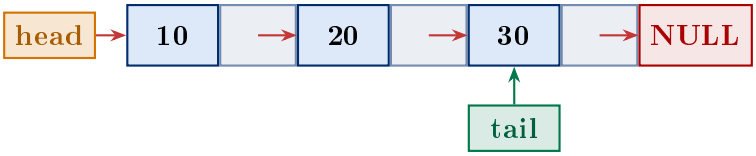
Linked Lists

From Absolute Basics to Advanced Mastery

Complete Study Notes with Your Own C++ Code



Files Covered in This Document

- `singlyLinkedList.cpp`
- `Assignment3Ques1.cpp`
- `Assignment3Ques2.cpp`
- `Assignment3Ques3.cpp`
- `Assignment3Ques4.cpp`
- `DoublyLinkedList.cpp`

github.com/SouvikGhorui/Data-Structure-and-Algorithm-with-CPP

Contents

1 What is a Linked List? 3
- 1.1 The Core Idea . . . 3
- 1.2 Your Node Class . . . 3
- 1.3 Your LinkedList Class Structure . . . 4
- 1.4 Why Two Pointers (head AND tail)? . . . 4

2 Core File: singlyLinkedList.cpp 5
- 2.1 push_front – Insert at Beginning . . . 5
- 2.2 push_back – Insert at End . . . 6
- 2.3 pop_front – Remove from Beginning . . . 6
- 2.4 pop_back – Remove from End . . . 7
- 2.5 insert – Insert at Any Position . . . 7
- 2.6 show – Display the List . . . 8
- 2.7 search – Find a Value . . . 8

3 Assignment3Ques1.cpp – Search & Length 10
- 3.1 search – Boolean Version . . . 10
- 3.2 length – Count Nodes . . . 10
- 3.3 Complete main() for Ques1 . . . 11

4 Assignment3Oues2.cpp – Deletion Methods 12
- 4.1 pop_front and pop_back . . . 12
- 4.2 pop(x) – Delete Node by Value . . . 12
- 4.3 Complete main() for Oues2 . . . 13

5 Assignment3Ques3.cpp – Reverse & Middle 14
- 5.1 ReverseLL – Recursive Reverse Print . . . 14
- 5.2 middleEle – Find the Middle . . . 15

6 Assignment3Ques4.cpp – Remove & Re-insert 17
- 6.1 insert(pos, Node*) – Insert Using a Node Pointer . . . 17
- 6.2 remove_insert – Move a Node from One Position to Another . . . 17

7 DoublyLinkedList.cpp – Doubly Linked List 19
- 7.1 What is a Doubly Linked List? . . . 19
- 7.2 Your Node Class for DLL . . . 19
- 7.3 push_front – DLL version . . . 20
- 7.4 push_back – DLL version . . . 20
- 7.5 pop_front – DLL version . . . 20
- 7.6 pop_back – DLL version: now truly O(1)! . . . 21

7.7 show – DLL version 21
7.8 Complete DLL main() 22

8 SLL vs DLL – Comparison from Your Code 23
8.1 Operations Compared 23
8.2 Structural Comparison 23

9 Advanced Operations 24
9.1 Actual List Reversal – Three Pointer Technique 24
9.2 Floyd's Cycle Detection 24
9.3 Fast Middle Element (No Length Needed) 25
9.4 Merge Sort on Linked List 25

10 Bug Summary and Fixes 27
10.1 All Bugs Found in Your Files 27
10.2 Fixed pop() – Safe Version 27
10.3 Fixed search() – Correct Position Counter 28

11 Complete Complexity Reference 29
11.1 All Your Methods at a Glance 29

1. What is a Linked List?

1.1 The Core Idea

A **Linked List** is a linear data structure made of **nodes**. Each node stores two things:

1. **data** – the actual integer (or any value) you want to store
2. **next** – a pointer to the next node in the chain

The last node's `next` is `NULL` – meaning "nothing comes after."

Think of a linked list like a treasure hunt – each clue (node) tells you where the next clue (node) is. The last clue says "the treasure hunt is over" (`NULL`).

1.2 Your Node Class

In every file in your repo, you define a `Node` class like this:

```cpp
class Node {
public:
    int data;     // stores the value
    Node* next;   // stores address of the next node

    Node(int val) {
        data = val;
        next = NULL;    // you use NULL (C-style), which is fine in C++
    }
};
```

Listing 1.1: Node class – your style

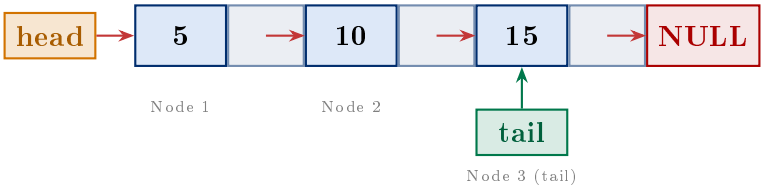
Note

You use `NULL` throughout your code. This is perfectly valid C++. Modern C++ (C++11 onwards) introduces `nullptr` which is type-safe, but `NULL` works correctly

for pointer comparisons.

1.3 Your LinkedList Class Structure

All your linked list classes carry both a `head` and a `tail` pointer. This is a smart design – it lets `push_back` run in $O(1)$ instead of $O(n)$!




```cpp
class LinkedList {
public:
    Node* head;
    Node* tail;

    LinkedList() {
        head = tail = NULL;  // empty list: both point to nothing
    }
    // ... methods below
};
```

Listing 1.2: LinkedList class with head and tail

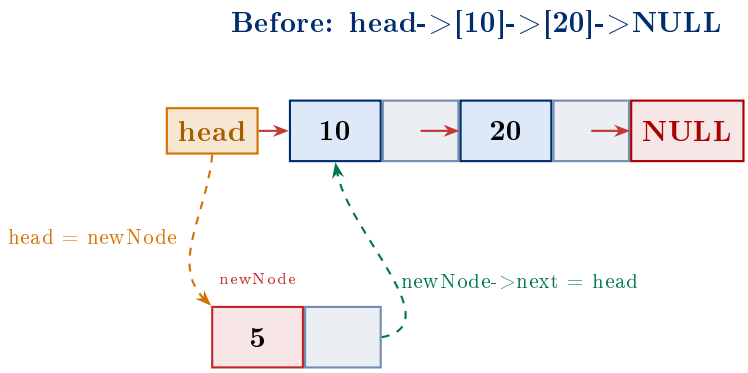
1.4 Why Two Pointers (head AND tail)?

Operation	Only head	head + tail
`push_front`	$O(1)$	$O(1)$
`push_back`	$O(n)$ – must walk to end	$O(1)$ – tail points there
`pop_back`	$O(n)$	$O(n)$ – still need to find second-to-last
`show`	$O(n)$	$O(n)$

2. Core File: singlyLinkedList.cpp

This is your foundational file. It implements all basic operations on a singly linked list. Let's go through every method.

2.1 push_front – Insert at Beginning

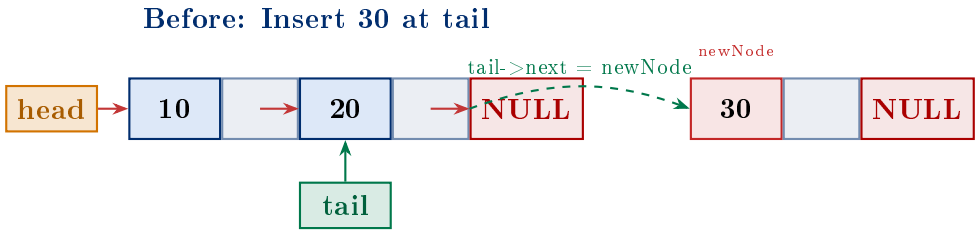



```cpp
void push_front(int val) {  // O(1)
    Node* newNode = new Node(val);
    if (head == NULL) {
        head = tail = newNode;  // first node: both head and tail point to it
        return;
    }
    else {
        newNode->next = head;   // new node points to old head
        head = newNode;         // head is updated to new node
    }
}
```

Listing 2.1: push_front – O(1)

Time: $O(1)$ – no traversal needed, just update head.
Space: $O(1)$ – only one new node allocated.

2.2 push_back – Insert at End




```cpp
void push_back(int val) {  // O(1)
    Node* newNode = new Node(val);
    if (head == NULL) {
        head = tail = newNode;  // empty list
    }
    else {
        tail->next = newNode;   // old tail links to new node
        tail = newNode;         // update tail to new node
    }
}
```

Listing 2.2: push_back – O(1) thanks to tail pointer

Because you store a `tail` pointer, `push_back` is $O(1)$. Without `tail`, you would walk the entire list each time – $O(n)$.

2.3 pop_front – Remove from Beginning

```cpp
void pop_front() {  // O(1)
    if (head == NULL) {
        cout << "There is nothing to delete!!";
        return;
    }
    Node* temp = head;        // save old head
    head = head->next;        // move head to next node
    temp->next = NULL;        // disconnect old head
    delete temp;              // free memory
}
```

Listing 2.3: pop_front – O(1)

2.4 pop_back – Remove from End

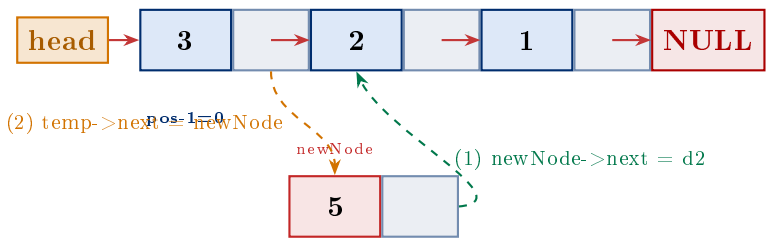
```cpp
void pop_back() {  // O(n)
    if (head == NULL) {
        cout << "There is nothing to delete!!";
        return;
    }
    Node* temp = head;
    while (temp->next != tail) {   // walk until node BEFORE tail
        temp = temp->next;
    }
    temp->next = NULL;   // second-to-last becomes new tail
    delete tail;
    tail = temp;
}
```

Listing 2.4: pop_back – O(n) for singly linked list

Why is pop_back O(n) even with a tail pointer? We know where the `tail` is, but we need to find the node *before* it to update its `next` to `NULL`. In a singly linked list, there is no `prev` pointer, so we must walk from `head`. This is why doubly linked lists (your `DoublyLinkedList.cpp`) solve this.

2.5 insert – Insert at Any Position

insert(5, pos=1): [3]->[2]->[1] – insert 5 after index 0



Source: `singlyLinkedList.cpp`

```cpp
void insert(int val, int pos) {  // O(n)
    if (pos < 0) {
        return;
    }
    else if (pos == 0) {
        push_front(val);          // reuse push_front for position 0
    }
    else {
```

```
        Node* newNode = new Node(val);
        Node* temp = head;
        for (int i = 0; i < pos - 1; i++) {   // walk to pos -1
            if (temp == NULL) {
                cout << "Invalid pos";
                return;
            }
            temp = temp->next;
        }
        newNode->next = temp->next;   // (1) link new node forward
        temp->next = newNode;         // (2) link prev node to new
    }
}
```

Listing 2.5: insert at position – O(n)

2.6 show – Display the List

```
void show() {  // O(n)
    Node* temp;
    temp = head;
    while (temp != NULL) {
        cout << temp->data << " -> ";   // print each node
        temp = temp->next;
    }
    cout << "NULL" << endl;  // marks end of list
}
// Output: 3 -> 5 -> 2 -> 1 -> NULL
```

Listing 2.6: show – O(n)

2.7 search – Find a Value

Source: singlyLinkedList.cpp

```
void search(int val) {  // O(n)
    Node* temp = head;
    while (temp != NULL) {
        int count = 0;                  // BUG: count is reset every iteration!
        if (temp->data == val) {
            cout << "Value is in the " << count << " th
```

```
    place" << endl;
            return;
        }
        temp = temp->next;
        count++;
    }
    cout << "Value is not in linkedlist" << endl;
}
```

Listing 2.7: search – O(n)

Bug in `search()` – `count` is declared inside the loop!
`int count = 0;` is inside the `while` loop, so it resets to `0` on every iteration. It will always print "position 0" even if the value is at position 5.
Fixed version:

```
void search(int val) {
    Node* temp = head;
    int count = 0;          // declare OUTSIDE the loop
    while (temp != NULL) {
        if (temp->data == val) {
            cout << "Value found at position " << count <<
    endl;
            return;
        }
        temp = temp->next;
        count++;            // only increment, never reset
    }
    cout << "Value not in linkedlist" << endl;
}
```

3. Assignment3Ques1.cpp – Search & Length

This file adds two important utility methods: `search` (returns `bool`) and `length`.

3.1 search – Boolean Version

This version is cleaner – it takes the `head` pointer as a parameter and returns `true`/`false`.

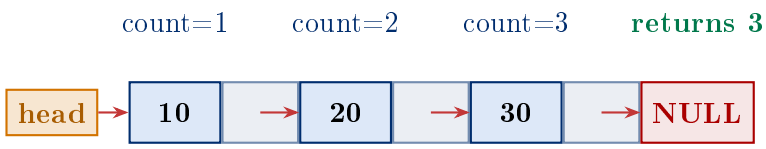
```cpp
bool search(Node* head, int x) {
    Node* temp = head;
    while (temp != NULL) {
        if (temp->data == x) {
            return true;     // found!
        }
        temp = temp->next;
    }
    return false;            // not found
}
```

Listing 3.1: search – boolean return, O(n)

This version is better than the one in `singlyLinkedList.cpp` because:

- Returns `bool` so the caller can use it in `if` statements
- No bug with `count`
- `main()` shows correct usage: `if(LL.search(LL.head, 4)) { ... }`

3.2 length – Count Nodes



```cpp
int length(Node* head) {
    Node* temp = head;
    int count = 0;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}
```

Listing 3.2: length – count nodes, O(n)

3.3 Complete main() for Ques1

```cpp
int main() {
    int n, temp;
    cout << "Enter number of elements: ";
    cin >> n;
    LinkedList LL;
    for (int i = 0; i < n; i++) {
        cin >> temp;
        LL.push_back(temp);
    }
    if (LL.search(LL.head, 4)) {
        cout << "4 is in LinkedList" << endl;
    }
    LL.show();
    cout << "Total nodes = " << LL.length(LL.head) << endl;
    return 0;
}
```

Listing 3.3: main() – Ques1

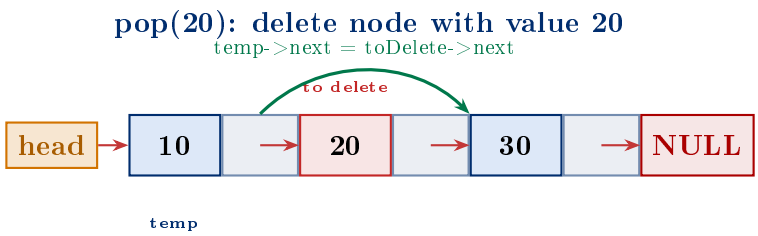
4. Assignment3Oues2.cpp – Deletion Methods

This file focuses on deletion: `pop_front`, `pop_back`, and `pop(x)` (delete by value).

4.1 pop_front and pop_back

Same as in `singlyLinkedList.cpp`. Covered in Chapter 2.

4.2 pop(x) – Delete Node by Value



Source: `Assignment3Oues2.cpp`

```cpp
void pop(int x) {
    if (head == NULL) {
        return;
    }
    // Special case: value is at head
    if (head->data == x) {
        pop_front();
        return;
    }
    Node* temp = head;
    // Walk until temp->next has the value x
    while ((temp->next->data != x) && (temp->next != NULL)) {
        temp = temp->next;
    }
    if (temp->next == NULL) {
        return;   // value not found
    }
    // Special case: value is at tail
    if (temp->next == tail) {
```

```
        pop_back();
        return;
    }
    // General case: unlink and delete
    Node* toDelete = temp->next;
    temp->next = toDelete->next;
    delete toDelete;
    return;
}
```

Listing 4.1: pop(x) – delete node by value, O(n)

Potential Crash in the `while` condition!
The condition is:

```
while ((temp->next->data != x) && (temp->next != NULL))
```

If `temp->next` is `NULL` and we evaluate `temp->next->data`, the program will crash with a **segmentation fault** (null pointer dereference). C++ evaluates left-to-right, so `temp->next->data` runs before the `NULL` check.
Fixed version – swap the condition order:

```
while (temp->next != NULL && temp->next->data != x)
//      ^^^ check NULL first ^^^
```

4.3 Complete main() for Oues2

```
int main() {
    int n, temp;
    cout << "Enter number of elements: ";
    cin >> n;
    LinkedList LL;
    for (int i = 0; i < n; i++) {
        cin >> temp;
        LL.push_back(temp);
    }
    LL.show();
    LL.pop(3);      // delete node containing value 3
    LL.show();
    return 0;
}
```

Listing 4.2: main() – Oues2

5. Assignment3Ques3.cpp – Reverse & Middle

This file implements two classic interview problems: printing the list in reverse (recursively) and finding the middle element.

5.1 ReverseLL – Recursive Reverse Print

```cpp
void ReverseLL(Node* head) {
    if (head == NULL) {
        return;            // base case: end of list
    }
    ReverseLL(head->next);  // recurse FIRST (go to the end)
    cout << head->data << " ";  // print on the WAY BACK
}
```

Listing 5.1: ReverseLL – recursive reverse print, O(n)

How recursion prints in reverse – call stack visualised:

List: 1 -> 2 -> 3 -> NULL

Call 1: ReverseLL(1) – calls ReverseLL(2) first

Call 2: ReverseLL(2) – calls ReverseLL(3) first

Call 3: ReverseLL(3) – calls ReverseLL(NULL)

Call 4: ReverseLL(NULL) – base case, RETURN

Unwinding: prints 3, then 2, then 1 => output: 3 2 1

`ReverseLL()` **does not actually reverse the list**. It only *prints* values in reverse order. The original list stays unchanged. To actually reverse the list (modify pointers), you need the three-pointer iterative approach shown in Chapter 8.

Time: $O(n)$ – every node visited once.
Space: $O(n)$ – recursion uses the call stack. One stack frame per node.

5.2 middleEle – Find the Middle

Your approach: compute the full length, then walk to `length/2`.

List: 10->20->30->40->50, length=5, length/2=2, middle=30

```cpp
int lengthLL(Node* head) {
    Node* temp = head;
    if (temp == NULL) return 0;
    int count = 0;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}

void middleEle(Node* head) {
    Node* temp = head;
    for (int i = 0; i < lengthLL(temp) / 2; i++) {
        temp = temp->next;     // advance lengthLL/2 steps
    }
    cout << "Middle element = " << temp->data;
}
```

Listing 5.2: middleEle and lengthLL – O(n)

Complexity Analysis

Your approach: $O(n)$ for `lengthLL` + $O(n/2)$ for the walk = $O(n)$ total. Calls `lengthLL` once per call to `middleEle`.
Faster single-pass approach (Floyd's tortoise and hare):

```cpp
void middleEle_fast(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
```

```
            fast = fast->next->next;
        }
        cout << "Middle = " << slow->data;
    }
```

One pass, no extra function call – still $O(n)$ but runs faster in practice.

6. Assignment3Ques4.cpp – Remove & Re-insert

This file introduces two new ideas: `push_front`, `insert` (taking a `Node*`), and the interesting `remove_insert` function.

6.1 insert(pos, Node*) – Insert Using a Node Pointer

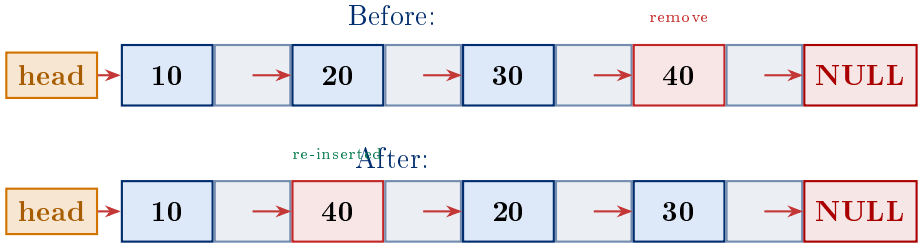
```
void insert(int pos, Node* node) {
    int count = 0;
    Node* temp = head;
    if (pos == 0) {
        push_front(node->data);
        return;
    }
    while (count != pos - 1) {      // walk to pos-1
        temp = temp->next;
        count++;
    }
    node->next = temp->next;        // link new node forward
    temp->next = node;              // link prev node to new
}
```

Listing 6.1: insert – takes position and Node pointer

6.2 remove_insert – Move a Node from One Position to Another

This is a unique function you wrote. It removes a node from `remove_pos` and re-inserts it at `insert_pos`.

remove_insert(3, 1): remove index 3, insert at index 1




```cpp
void remove_insert(int remove_pos, int insert_pos) {
    Node* temp = head;
    int count = 0;

    // Step 1: walk to node BEFORE remove_pos
    if (remove_pos == 0) {
        head = head->next;
    }
    else {
        while (count != remove_pos - 1) {
            temp = temp->next;
            count++;
        }
    }

    // Step 2: save the node to be moved, then unlink it
    Node* tempNode = new Node(temp->next->data);  // copy the data
    Node* temp2    = temp->next;
    temp->next     = temp2->next;
    delete temp2;

    // Step 3: re-insert at the desired position
    insert(insert_pos, tempNode);
}
```

Listing 6.2: remove_insert – O(n)

How remove_insert works step by step:

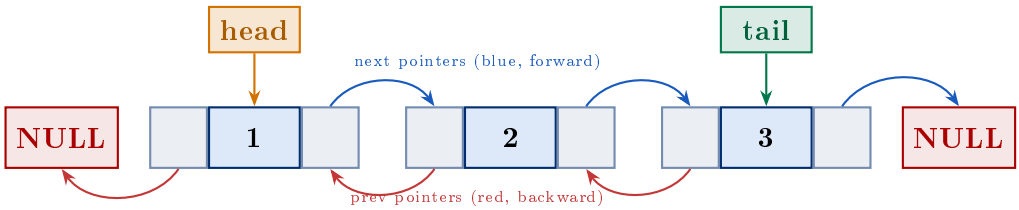
1. Walk to the node at `remove_pos - 1` (one before the target).
2. Copy the target node's data into a new node (`tempNode`).
3. Unlink the original node from the list and `delete` it.
4. Call `insert(insert_pos, tempNode)` to place the copy at the new position.

7. DoublyLinkedList.cpp – Doubly Linked List

7.1 What is a Doubly Linked List?

Your `DoublyLinkedList.cpp` introduces a `Node` with *three* fields: `prev`, `data`, and `next`. This lets you traverse both forwards and backwards.

Your DLL: each node has prev and next pointers



7.2 Your Node Class for DLL

```cpp
class Node {
public:
    int data;
    Node* next;
    Node* prev;          // NEW: backward pointer

    Node(int val) {
        data = val;
        next = NULL;
        prev = NULL;
    }
};
```

Listing 7.1: DLL Node with prev and next

7.3 push_front – DLL version

```cpp
void push_front(int val) {
    Node* newNode = new Node(val);
    if (head == NULL) {
        head = tail = newNode;
    }
    else {
        newNode->next = head;     // new node's next = old head
        head->prev    = newNode; // old head's prev = new node
        head          = newNode; // update head
    }
}
```

Listing 7.2: DLL push_front – O(1)

7.4 push_back – DLL version

```cpp
void push_back(int val) {
    Node* newNode = new Node(val);
    if (head == NULL) {
        head = tail = newNode;
    }
    else {
        newNode->prev = tail;     // new node's prev = old tail
        tail->next    = newNode; // old tail's next = new node
        tail          = newNode; // update tail
    }
}
```

Listing 7.3: DLL push_back – O(1)

7.5 pop_front – DLL version

Source: DoublyLinkedList.cpp

```cpp
void pop_front() {
    if (head == NULL) return;
    Node* temp = head;
    head = head->next;
```

```
    if (head != NULL) {
        head->prev = NULL;     // new head has no previous node
    }
    temp->next = NULL;
    delete temp;
}
```

Listing 7.4: DLL pop_front – O(1)

7.6 pop_back – DLL version: now truly O(1)!

```
void pop_back() {
    if (head == NULL) return;
    Node* temp = tail;
    tail = tail->prev;          // move tail back using prev pointer
    if (tail != NULL) {
        tail->next = NULL;      // new tail has no next
    }
    temp->prev = NULL;
    delete temp;
}
```

Listing 7.5: DLL pop_back – O(1) thanks to prev pointer!

This is the key advantage of DLL! In `singlyLinkedList.cpp`, `pop_back` is $O(n)$ because you must walk to find the second-to-last node. In DLL, `tail->prev` gives you the second-to-last node directly in $O(1)$.

7.7 show – DLL version

Source: `DoublyLinkedList.cpp`

```
void show() {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << "<=>";    // your style: <=> separator
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
```

```
// Output: 1<=>2<=>3<=>NULL
```

Listing 7.6: DLL show

7.8 Complete DLL main()

```
int main() {
    doublyList dll;
    dll.push_back(1);
    dll.push_back(2);
    dll.push_back(3);
    dll.pop_back();        // removes 3
    dll.show();            // Output: 1<=>2<=>NULL
    return 0;
}
```

Listing 7.7: main() – DoublyLinkedList.cpp

8. SLL vs DLL – Comparison from Your Code

8.1 Operations Compared

Operation	Your SLL	Your DLL	Why different?
`push_front`	$O(1)$	$O(1)$	Same logic
`push_back`	$O(1)$	$O(1)$	Both use `tail` pointer
`pop_front`	$O(1)$	$O(1)$	Same logic
`pop_back`	$O(n)$	$O(1)$	DLL has `prev` pointer
`insert`	$O(n)$	$O(n)$	Must search
`search`	$O(n)$	$O(n)$	Must traverse
`show`	$O(n)$	$O(n)$	Full traversal
Memory per node	1 ptr	2 ptrs	DLL uses more RAM

8.2 Structural Comparison

Property	SLL (your files)	DLL (your file)
Class name	`LinkedList` / `List`	`doublyList`
Node fields	`data, next`	`data, next, prev`
Separator in `show`	`" -> "`	`"<=>"`
Backward traversal	No	Yes
Delete without search	No	Yes (if you have the pointer)

9. Advanced Operations

9.1 Actual List Reversal – Three Pointer Technique

Your `ReverseLL` in Ques3 only *prints* in reverse. This version actually *modifies* the list using your coding style:

```
void reverseList() {
    Node* prev = NULL;
    Node* curr = head;
    Node* next = NULL;
    tail = head;            // old head becomes new tail

    while (curr != NULL) {
        next        = curr->next;   // save next
        curr->next  = prev;         // flip pointer
        prev        = curr;         // advance prev
        curr        = next;         // advance curr
    }
    head = prev;            // old tail becomes new head
}
```

Listing 9.1: True reverse – modifies the list

Dry run on your list [1 → 2 → 3 → NULL]:

Step	prev	curr	next	What happens
Start	NULL	1	–	initialise
1	NULL	1	2	1->NULL; prev=1; curr=2
2	1	2	3	2->1; prev=2; curr=3
3	2	3	NULL	3->2; prev=3; curr=NULL
End	3	NULL	–	head = 3

9.2 Floyd's Cycle Detection

```
bool hasCycle() {
    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
```

```
        slow = slow->next;          // 1 step
        fast = fast->next->next;    // 2 steps

        if (slow == fast) return true;  // met inside cycle
    }
    return false;
}
```

Listing 9.2: Cycle detection – two pointer technique

9.3 Fast Middle Element (No Length Needed)

Improvement over your `middleEle` in Ques3:

```
void middleEle_fast(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;          // 1 step
        fast = fast->next->next;    // 2 steps
    }
    // when fast hits end, slow is at middle
    cout << "Middle element = " << slow->data;
}
```

Listing 9.3: Fast middle – single pass, no lengthLL call

Your version: $O(n)$ for `lengthLL` + $O(n/2)$ walk = **1.5n** steps.
Fast version: Single pass, exactly **n/2** steps. Same $O(n)$ class but twice as fast in practice.

9.4 Merge Sort on Linked List

```
Node* getMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head->next;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;    // slow is at mid
}

Node* merge(Node* a, Node* b) {
    if (a == NULL) return b;
    if (b == NULL) return a;
    if (a->data <= b->data) {
```

```
        a->next = merge(a->next, b);
        return a;
    }
    b->next = merge(a, b->next);
    return b;
}

Node* mergeSort(Node* head) {
    if (head == NULL || head->next == NULL)
        return head;
    Node* mid        = getMiddle(head);
    Node* secondHalf = mid->next;
    mid->next        = NULL;            // split
    return merge(mergeSort(head), mergeSort(secondHalf));
}
// Usage: LL.head = mergeSort(LL.head);
```

Listing 9.4: Merge Sort – O(n log n)

10. Bug Summary and Fixes

10.1 All Bugs Found in Your Files

File	Function	Bug	Fix
singlyLinkedList	`search()`	`int count = 0` inside loop, resets every iteration	Move `int count = 0` to before the `while` loop
Assignment3Oues2	`pop(x)`	`temp->next->data` evaluated before `NULL` check – crash risk	Swap condition order to check `temp->next != NULL` first
Assignment3Ques4	`remove_insert()`	When `remove_pos == 0` the code runs `head = head->next` but the original head is not deleted – memory leak	Save old head, then delete it
Assignment3Ques3	`ReverseLL()`	Comment in `main()` says "ReverseLL" but it is commented out – not called	Un-comment to use it, and note it only prints, does not reverse

10.2 Fixed pop() – Safe Version

```
void pop(int x) {
    if (head == NULL) return;

    if (head->data == x) {
        pop_front();
        return;
    }

    Node* temp = head;
    // FIX: check NULL before accessing ->data
    while (temp->next != NULL && temp->next->data != x) {
        temp = temp->next;
    }

```

```
    if (temp->next == NULL) return;    // not found

    if (temp->next == tail) {
        pop_back();
        return;
    }

    Node* toDelete = temp->next;
    temp->next = toDelete->next;
    delete toDelete;
}
```

Listing 10.1: pop(x) – fixed safe version

10.3 Fixed search() – Correct Position Counter

```
void search(int val) {
    Node* temp  = head;
    int   count = 0;            // FIX: declare OUTSIDE the while loop

    while (temp != NULL) {
        if (temp->data == val) {
            cout << "Value is in the " << count << " th place" << endl;
            return;
        }
        temp = temp->next;
        count++;
    }
    cout << "Value is not in linkedlist" << endl;
}
```

Listing 10.2: search() – fixed, count declared outside loop

11. Complete Complexity Reference

11.1 All Your Methods at a Glance

Method	File	Time	Space
`push_front`	SLL files	$O(1)$	$O(1)$
`push_back`	SLL files	$O(1)$	$O(1)$
`pop_front`	SLL files	$O(1)$	$O(1)$
`pop_back`	SLL files	$O(n)$	$O(1)$
`pop_back`	DoublyLinkedList	$O(1)$	$O(1)$
`insert`	SLL files	$O(n)$	$O(1)$
`pop(x)`	Oues2	$O(n)$	$O(1)$
`search`	Ques1, SLL	$O(n)$	$O(1)$
`length/lengthLL`	Ques1, Ques3	$O(n)$	$O(1)$
`ReverseLL`	Ques3	$O(n)$	$O(n)$
`middleEle`	Ques3	$O(n)$	$O(1)$
`remove_insert`	Ques4	$O(n)$	$O(1)$
`show`	All files	$O(n)$	$O(1)$